

Low-Level Design (LLD) & Class Diagrams

Project: Offline-First, Proxy-Free College Attendance System

This document outlines the Object-Oriented Low-Level Design (LLD) of the system. It defines the core domain models (entities), the backend service layer (FastAPI controllers), and the client-side managers (PWA/WASM).

1. Core Domain Models (Entity Layer)

This Class Diagram represents the database entities mapped via SQLAlchemy (Python ORM). It shows how Users, Courses, Sessions, and Attendance Logs relate to one another.

UML Class Diagram (Mermaid)

```
classDiagram
    %% --- Inheritance ---
    User <|-- Student
    User <|-- Professor
    User <|-- Admin

    %% --- Classes ---
    class User {
        <<Abstract>>
        +UUID id
        +String name
        +String email
        +String password_hash
        +Enum role
        +login(email, password)
    }

    class Student {
        +String roll_number
        +List~Float~ face_embedding_128d
        +requestDeviceReset()
        +getAttendanceStats()
    }

    class Professor {
        +String department
        +startSession(Course)
        +exportReport(Session)
        +overrideAttendance(Student, Session)
    }

    class Admin {
        +approveDeviceReset(Student)
        +uploadBulkRoster(File)
    }

    class Course {
        +String course_code
```

```

+String course_name
+String google_sheet_id
+getEnrollmentList()
}

class DeviceBinding {
+UUID device_id
+String webauthn_public_key
+String hardware_hash
+Boolean is_active
+validateSignature(payload)
}

class Session {
+UUID session_id
+DateTime start_time
+DateTime end_time
+String totp_secret_key
+generateDynamicTOTP()
+closeSession()
}

class AttendanceLog {
+UUID log_id
+DateTime scanned_at
+String status
+String crypto_signature
+Boolean is_synced
+verifyTimestamp()
}

%% --- Relationships ---
Student "1" -- "1..*" DeviceBinding : owns
Professor "1" -- "*" Course : teaches
Student "*" -- "*" Course : enrolled_in
Course "1" -- "*" Session : instances
Session "1" -- "*" AttendanceLog : contains
Student "1" -- "*" AttendanceLog : generates

```

2. Backend Service Layer (FastAPI Controllers)

These classes represent the business logic handlers in the FastAPI backend. They isolate the processing logic from the database models.

```

classDiagram
class AuthManager {
+registerWebAuthn(Student)
+verifyWebAuthnChallenge(Response)
+generateJWT(User)
+processResetRequest(Student)
}

class ValidationEngine {
+verifyTOTPNonce(nonce, session_secret)
+validateProfessorTimestamp(encrypted_time)
}

```

```

        +checkDuplicateSubmission(Student, Session)
    }

    class SyncService {
        +queueOfflinePayload(JSON)
        +batchCommitToDB(List~AttendanceLog~)
        +exportToGoogleSheets(Course, List~AttendanceLog~)
        +generateExcelReport(Session) File
    }

    ValidationEngine ..> SyncService : "Passes valid logs to"
    AuthManager ..> ValidationEngine : "Provides trusted User context"

```

3. Client-Side (PWA) Managers

Since the system relies heavily on offline browser capabilities, the frontend (React/TypeScript) is modeled with specific manager classes to handle hardware APIs.

```

classDiagram
    class BiometricScanner {
        -MediaStream cameraStream
        -WASMMModel faceApi
        +startCamera()
        +extract128DVector() FloatArray
        +runGazeLivenessChallenge() Boolean
    }

    class QREngine {
        -String sharedSecret
        -Interval timer
        +generateTOTP(currentTime) String
        +renderVisualNoise()
    }

    class OfflineStorageManager {
        -IndexedDB db
        +saveEncryptedPayload(Payload)
        +getPendingSyncs() List
        +triggerBackgroundSync()
    }

    BiometricScanner --> OfflineStorageManager : "Sends verified scan"

```

4. Design Patterns Utilized

To score high marks in a System Design course, mention these software design patterns implemented in your LLD:

1. **Singleton Pattern:** Used for the `DatabaseConnectionPool` and `OfflineStorageManager` (`IndexedDB`) to ensure only one instance handles read/writes, preventing database locks or race conditions.

2. **Strategy Pattern:** Used in the `SyncService` . The system can switch dynamically between `GoogleSheetsExportStrategy` and `ExcelDownloadStrategy` depending on the professor's configuration.
3. **Factory Method:** Used to instantiate different types of `User` (Student, Professor, Admin) during the bulk CSV onboarding process, ensuring each role gets the correct default permissions.